Applications in Digital Signal Processing Principles to Simulate and Design Communication and Imaging Systems

Dela Chica, Kyle EJ C.
Faculty of Engineering
University of Santo Tomas
España, Manila
kyleejdelachica.eng@ust.edu.ph

Maballo, Ralph Vincent E.
Faculty of Engineering
University of Santo Tomas
España, Manila
ralphvincent.maballo.eng@ust.edu.ph

Mananguit, Jared Asher N.
Faculty of Engineering
University of Santo Tomas
España, Manila
jaredasher.mananguit.eng@ust.edu.ph

Pangan, Raven G.
Faculty of Engineering
University of Santo Tomas
España, Manila
raven.pangan.eng@ust.edu.ph

***Abstract* - This project aims to apply different topics and theories learned from the course in designing and analyzing 3 different applications. The implementation and simulation is done through MATLAB. Each application is plotted, observed, and analyzed in the time and frequency domain. The applications cover different topics that are crucial in digital signal processing (DSP).**

Index Terms - digital signal processing, oscillator, impulse, FFT, sidelobes

Introduction

Digital Signal Processing (DSP) is a cornerstone of electronics engineering. By transforming and analyzing signals, it allows us to solve complex and real - world problems. This project utilizes MATLAB as the tool in translating theoretical topics into practical applications. By demonstrating how theories can be implemented into algorithms, it allows us to better understand the theories taught and appreciate the fundamentals in digital signal processing.

A digital oscillator is a marginally stable infinite impulse response (IIR) filter. By placing the poles of the system exactly on the unit circle in the Z-plane, the system generates a sustained oscillation rather than decaying to zero.

To compute the spectrum of either a continuous-time or discrete-time signal, the values of the signal for all time are required. However, in practice, we observe signals for only a finite duration. Consequently, the spectrum of a signal can only be approximated from a finite data record in frequency analysis using the DFT [1]. This can be done through a process called windowing, it is done to capture a portion of a signal by multiplying it to a window function. Through this, it will result in a signal with a finite duration which lasts depending on the length of the window function. A basic example of a window function is the *Rectangular Window*. This process introduces a phenomenon called *leakage*, which is caused by the abrupt on-off switching of window functions such as the rectangular window. The leakage manifests in the form of unwanted ripples in the frequency domain which are called *sidelobes* [1]. It is important to reduce these sidelobes because they can cause distortion in weaker components of the signal. One way of doing so is by using different kinds of window functions, specifically the *Bartlett Window,* the *Hanning Window,* the *Hamming Window,* and the *Blackman - Harris Window* which will be explored more deeply in this project.

Digital filters are what make DSP prevalent in the field of electronics and communications. Essentially, filters can be used to either clean up a noisy signal or repair a damaged one. In practice, designing a filter involves defining specific goals for its characteristic whether it filters high or low frequency signals or signals within a specific range. The decision to use what type ultimately depends on the specific requirements and constraints of the project [1].

Digital Oscillator Design

We are tasked to design a digital oscillator that will produce the signal below:

$$y(n) = [cos(\tfrac{3\pi}{10}n) + cos(\tfrac{\pi}{2}n) + cos(\tfrac{3\pi}{4}n)]\,u(n)$$

This is a sum of three discrete-time sinusoids, all of which are causal and operate inside the unit circle. Using

the linearity principle, we can simplify the design and analysis by taking the Z-transform of each term. The input is an impulse; then the Z-transform of an Impulse is X(z) = 1, therefore

$$Y(z) = H(z) \cdot X(z)$$

Since X(z) = 1. The equation simplifies to:

$$Y(z) = H(z) \cdot 1$$

Then, we can say that:

$$Y(z) = H(z)$$

I. Implementation

Taking the Z-transform of each term allows us to model separate subsystems, one oscillator for each frequency, which are then summed to produce the final output. This method relies on standard-transform pairs, specifically:

$$x(n) = cos(w_o n)\, u(n)$$

We are tasked to implement a second-order system with the transfer function:

$$H(z) = \frac{1 - cos(\omega_0)\, z^{-1}}{1 - 2cos(\omega_0) z^{-1} + z^{-2}}$$

To verify the digital filter design, we are tasked with implementing the oscillator that follows.

- Parallel Second-Order System
- Cascaded Second-Order System
- Single Transfer Function

To obtain each subsystem, the three oscillators are identified.

$$H_1(z) = cos(\tfrac{3\pi}{10} n),\ \omega_1 = \tfrac{3\pi}{10}$$
$$H_2(z) = cos(\tfrac{\pi}{2} n),\ \omega_2 = \tfrac{\pi}{2}$$
$$H_3(z) = cos(\tfrac{3\pi}{4} n),\ \omega_3 = \tfrac{3\pi}{4}$$

The transform function for each subsystem,

$$H_1(z) = \frac{1 - z^{-1} cos(\frac{3\pi}{10})}{1 - 2z^{-1} cos(\frac{3\pi}{10}) + z^{-2}}$$

$$H_2(z) = \frac{1 - z^{-1} cos(\frac{\pi}{2})}{1 - 2z^{-1} cos(\frac{\pi}{2}) + z^{-2}}, \text{ since } cos(\tfrac{\pi}{2}) = 0$$

$$H_2(z) = \frac{1}{1 - z^{-2}}$$

$$H_3(z) = \frac{1 - z^{-1} cos(\frac{3\pi}{4})}{1 - 2z^{-1} cos(\frac{3\pi}{4}) + z^{-2}}$$

For this part, the coefficients of numerators and denominators of each subsystem are given by:

$$b_1 = [1, -cos(\tfrac{3\pi}{10})]$$
$$a_1 = [1, -2cos(\tfrac{3\pi}{10}), 1]$$

$$b_2 = [1, 0]$$
$$a_2 = [1, 0, 1]$$

$$b_3 = [1, -cos(\tfrac{3\pi}{4})]$$
$$a_3 = [1, -2cos(\tfrac{3\pi}{4}), 1]$$

MATLAB CODE:

```
% Number of samples
N = 50;
n = 0:N-1;
% Impulse input, x(n)=delta(n)
x = [1 zeros(1,N-1)];
% Frequencies of each subsystem
w1 = 3*pi/10;
w2 = pi/2;
w3 = 3*pi/4;
% Coefficients of subsystem 1
b1 = [1 -cos(w1)];
a1 = [1 -2*cos(w1) 1];
% Coefficients of subsystem 2
b2 = [1 -cos(w2)];
a2 = [1 -2*cos(w2) 1];
% Coefficients of subsystem 3
b3 = [1 -cos(w3)];
a3 = [1 -2*cos(w3) 1];
```

For Parallel-Second Order Subsystem

$$Y(z)_{parallel} = H_1(z) + H_2(z) + H_3(z)$$

$$Y(z)_{parallel} = [\frac{1 - z^{-1} cos(\frac{3\pi}{10})}{1 - 2z^{-1} cos(\frac{3\pi}{10}) + z^{-2}}] + [\frac{1}{1 - z^{-2}}] + [\frac{1 - z^{-1} cos(\frac{3\pi}{4})}{1 - 2z^{-1} cos(\frac{3\pi}{4}) + z^{-2}}]$$

For Cascaded Second-Order Subsystem

To implement the single transfer function H(z) constructed from cascaded second-order subsystems, we first need to integrate the three parallel components. This is achieved by finding a least common multiple for the denominators, allowing us to merge them into a unified expression.

$$Y(z)_{cascaded} = G \cdot C_1(z) \cdot C_2(z) \cdot C_3(z)$$

The denominator (A_{TOTAL}) of the poles of the system represents the frequencies the system can generate.

$$A(z)_{TOTAL} = D_1(z) \cdot D_2(z) \cdot D_3(z)$$

Then getting the specific values:

$$D_1 = 1 - 1.1755z^{-1} + z^{-2}$$
$$D_2 = 1 + z^{-2}$$
$$D_3 = 1 + 1.4142z^{-1} + z^{-2}$$

$$A(z)_{TOTAL} = 1 + 0.2387z^{-1} + 1.3376z^{-2} + 0.2387z^{-3} + 1.3376z^{-4} + 0.2387z^{-5} + z^{-6}$$

The numerator (B_{TOTAL}). To add fractions, we must cross-multiply:

$$H(z) = \frac{N_1}{D_1} + \frac{N_2}{D_2} + \frac{N_3}{D_3}$$
$$H(z) = \frac{N_1(D_2D_3) + N_2(D_1D_3) + N_3(D_1D_2)}{D_1D_2D_3}$$

Summing these gives the numerator

$$B(z)_{TOTAL} = 3 + 0.5967z^{-1} + 3.3374z^{-2} + 0.7161z^{-3} + 1.3374z^{-4} + 0.1193z^{-5}$$

To get the Cascade form, we must factor this fraction back into smaller pieces

We factor the denominator and numerator to find the roots (zeros) of the polynomial:

$$3 + 0.5967z^{-1} + 3.3374z^{-2} + 0.7161z^{-3} + 1.3374z^{-4} + 0.1193z^{-5} = 0$$

Since this is a 6th-degree polynomial, there is no simple formula that you can use. We use numerical algorithms (like roots() in MATLAB).

We can write the numerator as:

$$B(z)_{TOTAL} = G\,(1 - r_1z^{-1})\,(1 - r^*_1z^{-1})\,(1 - r_2z^{-1})\ldots$$

Now, we group the factors to create the Cascaded Subsystems:

$$H(z)_{cascade} = G \cdot \left[\frac{(1-r_1z^{-1})(1-r^*_1z^{-1})}{D_1(z)}\right] \cdot \left[\frac{(1-r_2z^{-1})(1-r^*_2z^{-1})}{D_2(z)}\right] \cdot \left[\frac{(1-r_3z^{-1})(1-r^*_3z^{-1})}{D_3(z)}\right]$$

$$H(z)_{cascade} = G \cdot \left[\frac{1 + b_{11}z^{-1} + b_{12}z^{-2}}{1 - 1.1755z^{-1} + z^{-2}}\right] \cdot \left[\frac{1 + b_{21}z^{-1} + b_{22}z^{-2}}{1 + z^{-2}}\right] \cdot \left[\frac{1 + b_{31}z^{-1} + b_{32}z^{-2}}{1 - 1.4142z^{-1} + z^{-2}}\right]$$

Implementation in MATLAB:

MATLAB CODE

```
%Calculate Total Denominator
(Convolution of all denominators)
A12 = conv(a1, aa2);
a_total = conv(a1, conv(a2, a3));

%Calculate Total Numerator
(Cross-multiplication sum)
term1 = conv(b1, conv(a2, a3));
term2 = conv(b2, conv(a1, a3));
term3 = conv(b3, conv(a1, a2));
B_total = term1 + term2 + term3;
```

In MATLAB, determining the total denominator (A_{TOTAL}) requires calculating the convolution of the individual denominator coefficients, such as a_1, a_2, and a_3. The total numerator (B_{TOTAL}) is obtained by convolving the numerator coefficients (b_1, b_2, b_3) of each branch with the denominators of the other branches and then summing those results together.

By merging these subsystems, the single transfer function becomes a 6th-order system. It is represented as:

$$H(z)_{TOTAL} = \frac{A(z)_{TOTAL}}{B(z)_{TOTAL}}$$

1. ***Implementing Parallel Second-Order Subsystem in MATLAB***

The parallel realization relies on the linearity property of the z-transform. In this configuration, the total transfer function is the sum of each transfer function.

MATLAB CODE:

```
% Simulate Parallel Branches
y_p1 = filter(b1, a1, x);
y_p2 = filter(b2, a2, x);
y_p3 = filter(b3, a3, x);
% Sum for Final Parallel Output
y_parallel = y_p1 + y_p2 + y_p3;
```

2. Implementing Cascaded Second-Order Subsystem in MATLAB:

To combine three small filter parts into one big system called H(z) in MATLAB, you first merge their bottom numbers using a convolution to find the total denominator. To find the top part, you multiply the top numbers of one branch by the bottom numbers of the others and add them all up, which results in a single 6th-order system. There are two main ways to set this up. In the parallel version, you send a signal through each small part separately and then add all the results together to get the final signal.

In the cascaded version, you connect the parts in a chain like a series. You use a special MATLAB command called "tf2sos" to break the big system into smaller pairs of poles and zeros. To test this, you simply pass the signal through these sections one after another until it reaches the end.

MATLAB CODE:

```
% Factoring the consolidated 6th-order
% transfer function
% into Second-Order Sections(SOS).
[sos_matrix, gain] = tf2sos(b_total,
a_total);
y_cascade = x * gain; % Applying the
overall gain
% Loop through each 2nd-order section
for k = 1:size(sos_matrix, 1)
b_k = sos_matrix(k, 1:3); % Extract
numerators
a_k = sos_matrix(k, 4:6); % Extract
denominators
% The input to this stage is the
output of the previous stage
y_cascade = filter(b_k, a_k,
y_cascade);
end
```

3. *Implementing Single Transfer Function in MATLAB*

To combine three small filter parts into one large system in MATLAB, you first merge their bottom numbers using a math process called convolution to find a single total denominator. To find the new top part, you multiply the top numbers of one branch by the bottom numbers of the other branches and add everything together, which creates a 6th-order system. There are three ways to set this up: (1) first, in the parallel method, you send a signal through each small part separately and add the results together. (2) Second, in the cascaded method, we use a tool called “tf2sos” to break the big system into smaller pairs and connect them in a series so the signal goes through them one after another. Finally, in the single transfer function method, you treat everything as one big fraction with a common denominator and filter the signal through it all at once.

MATLAB CODE:

```
y_stf = filter(b_total, a_total, x);
```

II. Analysis and Discussion

To check if the digital oscillator works, three different tests were done in MATLAB to look at how the system acts in different ways. First, a simple dot-and-line graph, called a stem plot, was used to look at the signal over time. This test proved that no matter how the system was built, whether it was set up in a parallel way, linked together in a cascaded way, or treated as one big single piece, the final signal always looked exactly the same.

Next, a second test used a tool to check the signal's frequency, which is like measuring how fast it vibrates. The resulting graph showed tall "peaks" at the exact same speeds that were asked for in this project, confirming the oscillator is running at the right frequencies.

Finally, a circular map called a pole-zero plot was used to check if the system stays steady and how all the mathematical parts are paired up. This map helps compare how the parts are grouped in the chain-like version versus the side-by-side version to make sure everything is organized correctly.

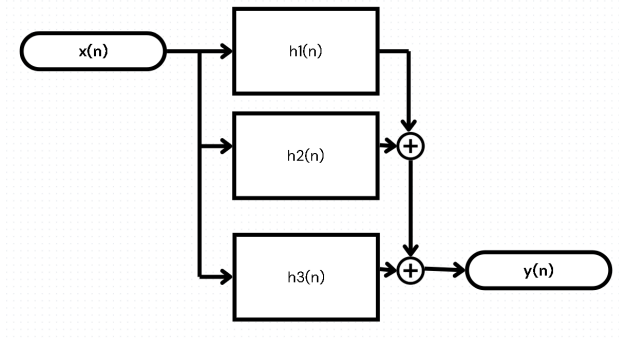

FIGURE 1.1. BLOCK DIAGRAM OF PARALLEL SECOND-ORDER SUBSYSTEM

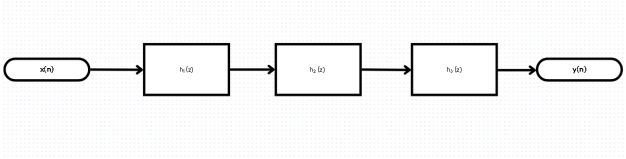

FIGURE 1.2. BLOCK DIAGRAM OF CASCADED SECOND-ORDER SUBSYSTEM

Plotting for Parallel, Cascaded, and Single Transfer Function Subsystem

MATLAB CODE:

```
% Time Domain Plots
% Parallel Second-Order Subsystem
figure;
stem(n,y_parallel,'filled');
```

```
xlabel('n'); ylabel('y(n)');
title('Parallel Realization Output
(Time Domain)');
% Cascaded Subsystems (The sections
from Second-Order Subsystem)
figure;
stem(n, y_cascade, 'filled');
xlabel('n'); ylabel('y(n)');
title('Cascaded Realization Output
(Time Domain)');
% Single Transfer Function (The 6th
Order System)
figure;
stem(n, y_stf, 'filled');
xlabel('n'); ylabel('y(n)');
title("Single Transfer Function
Output(Time Domain)");
grid on;
% Frequency Domain Plots
N = 1024;
Fs = 2;
% Frequency Vector
N_vector = (0:N/2-1)/(N/2-1)*Fs/2;
% Compute FFT (keep only the first
half)
Y_n = fft(y_stf, N);
Y_n = Y_n(1:N/2);
figure;
plot(N_vector, abs(Y_n));
title('Digital Oscillator Frequency
Response');
xlabel('Normalized Frequency (\pi
rad/sample)');
ylabel('|Y(\omega)|');
grid on;
%Pole-Zero Plots
% Parallel Second-Order Subsystem
figure;
subplot(1,3,1); zplane(b1, a1);
title('Subsystem1');
subplot(1,3,2); zplane(b2, a2);
title('Subsystem2');
subplot(1,3,3); zplane(b3, a3);
title('Subsystem3');
sgtitle('Pole-Zero Plots: Parallel
Second-Order Subsystem');
% Cascaded Subsystems (The sections
from Second-Order Subsystem)
figure;
subplot(1,3,1);
zplane(sos_matrix(1,1:3),
sos_matrix(1,4:6)); title('Cascade
Stage 1');
subplot(1,3,2);
zplane(sos_matrix(2,1:3),
sos_matrix(2,4:6)); title('Cascade
Stage 2');
subplot(1,3,3);
zplane(sos_matrix(3,1:3),
sos_matrix(3,4:6)); title('Cascade
Stage 3');
sgtitle('Pole-Zero Plot: Cascaded
Second-Order Subsystem');
% Single Transfer Function (The 6th
Order System)
figure;
zplane(b_total, a_total);
title('Pole-Zero Plot: Single Transfer
Function (6th Order)');
```

Time-Domain Response (with 50 samples)

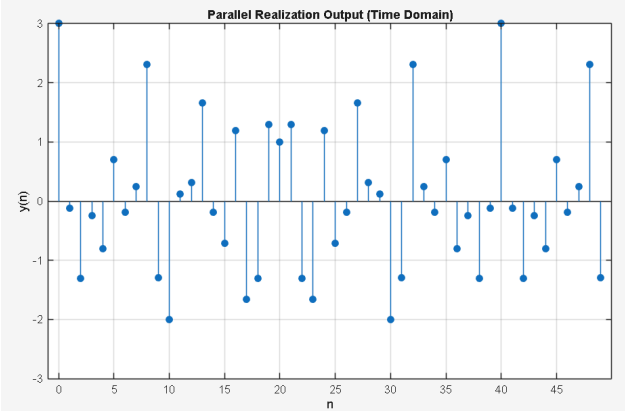

FIGURE 1.3. PARALLEL SECOND ORDER SUBSYSTEM

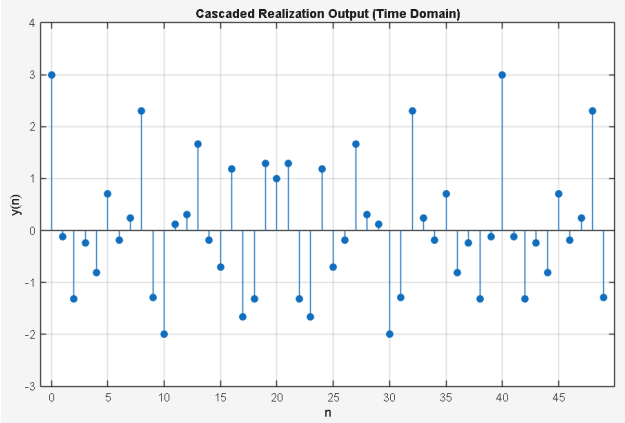

FIGURE 1.4. CASCADED SECOND ORDER SUBSYSTEM

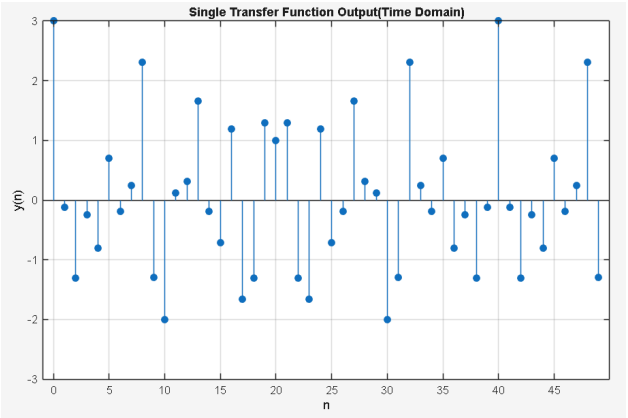

FIGURE 1.5. SINGLE TRANSFER FUNCTION

Frequency Response (with 50 samples)

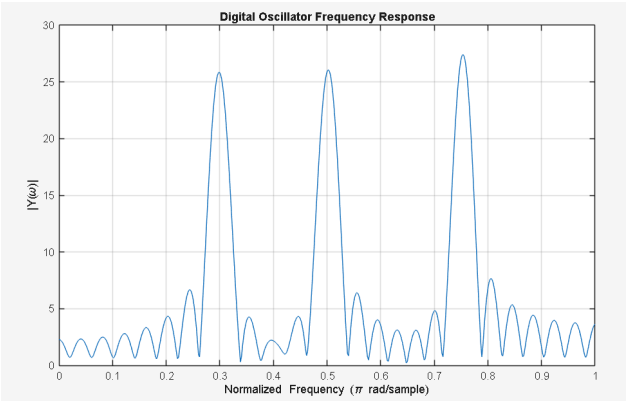

FIGURE 1.6. DIGITAL OSCILLATOR IN THE FREQUENCY DOMAIN

Pole-Zero Plot (with 50 samples)

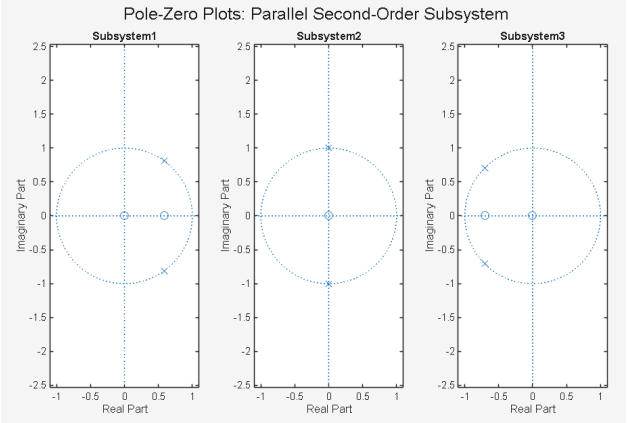

FIGURE 1.7. POLE-ZERO PLOTS OF PARALLEL SECOND-ORDER SUBSYSTEM

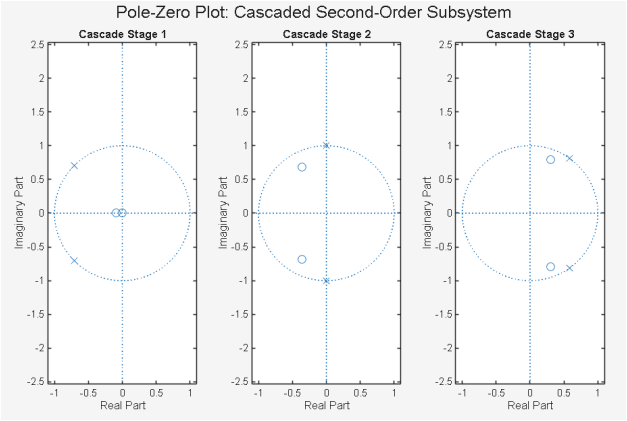

FIGURE 1.8. CASCADED SECOND ORDER SUBSYSTEM

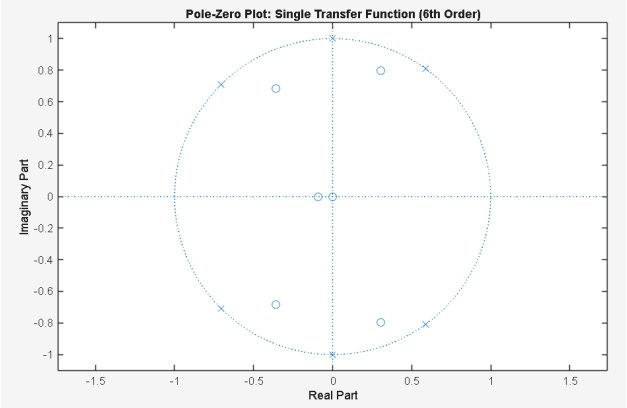

FIGURE 1.9. SINGLE TRANSFER FUNCTION IN 6TH ORDER

III. Conclusion

In conclusion, this project shows that there are several different ways to build a digital oscillator in MATLAB, but they all lead to the same result. By using the parallel, cascaded, and single transfer function methods, we were able to turn separate sine wave parts into one working system. Our tests in the time, frequency, and circle-map (z-domain) areas prove that the design is steady and vibrates at exactly the right speeds. This confirms that no matter which math structure is used, the system accurately creates the specific signal required for the design.

Window Function Evaluation

This section of the project evaluates the performances of five different window functions (Rectangular, Bartlett, Hanning, Hamming, and Blackman - Harris). And also, to determine how the window length can affect each window function.

I. Implementation

The evaluation begins with initializing the parameters needed for the simulation. The window length is set to L = 101 samples and the sample index is equal to n = 0 : L - 1. The sample index is for the plotting of each window function in the time domain. The generation of each window function sequence is done by making use of their formulas.

a. Rectangular Window

$$\omega(n) = \begin{cases} 1 & 0 \le n \le L-1 \\ 0 & elsewhere \end{cases}$$

b. Bartlett Window

$$\omega(n) = \begin{cases} 1 - \dfrac{2\left|n - \dfrac{L-1}{2}\right|}{L-1} & 0 \le n \le L-1 \\ 0 & elsewhere \end{cases}$$

c. Hanning Window

$$\omega(n) = \begin{cases} \dfrac{1}{2}\left(1 - \cos\left(\dfrac{2\pi n}{L-1}\right)\right) & 0 \le n \le L-1 \\ 0 & elsewhere \end{cases}$$

d. Hamming Window

$$\omega(n) = \begin{cases} 0.54 - 0.46\cos\left(\dfrac{2\pi n}{L-1}\right) & 0 \le n \le L-1 \\ 0 & elsewhere \end{cases}$$

e. Blackman - Harris Window

$$\omega(n) = \begin{cases} a_0 - a_1\cos\left(\dfrac{2\pi n}{L-1}\right) + a_2\cos\left(\dfrac{4\pi n}{L-1}\right) - a_3\cos\left(\dfrac{6\pi n}{L-1}\right) & 0 \le n \le L-1 \\ 0 & elsewhere \end{cases}$$

where $a_0 = 0.35875, a_1 = 0.48829, a_2 = 0.14128, a_3 = 0.01168$

MATLAB CODE:

```
%% Rectangular Window Function
% Parameters
L = 101;  % Window Length
n = 0 : L-1;  % Sample Index
% Implementation of the Rectangular
Window Function
w = ones(L,1);

%% Bartlett Window Function
% Implementation of the Bartlett
Window Function
N = 2 * abs(n - (L-1)/2);
w = 1 - N / (L - 1);

%% Hanning Window Function
% Implementation of the Hanning Window
Function
w = 0.5 * (1 - cos(2 * pi * n /
(L-1)));

%% Hamming Window Function
% Implementation of the Hamming Window
Function
w = 0.54 - 0.46 * cos(2 * pi * n /
(L-1));

%% Blackman - Harris Window Function
% Implementation of the Blackman -
Harris Window Function
a0 = 0.35875;
a1 = 0.48829 ; % Coefficient a1
a2 = 0.14128 ; % Coefficient a2
a3 = 0.01168 ; % Coefficient a3
% General Equation for the Blackman -
Harris Window
w = a0 - ...
   a1 * cos(2 * pi * n/ (L-1)) + ...
   a2 * cos(4 * pi * n/ (L-1)) - ...
   a3 * cos(6 * pi * n/ (L-1));
```

These sequences are then plotted in the time and frequency domain. The plotting of each window function in the frequency domain is done via a 1024 - point FFT for the smoothness. Which is then shifted to the center using the MATLAB function fftshift. This shifted sequence is then normalized to a peak of 1 (0 dB) and then converted to decibels using the formula 20 * log10. The horizontal axis for the frequency domain is in normalized form (0 to 1).

MATLAB CODE:

```
% Computation for the Frequency
Response
W = fft(w,1024); % 1024 - point FFT
W_shifted = fftshift(W); % Shift zero
frequency to center
% Normalize Magnitude
mag_W = abs(W_shifted);
% Normalizing to peak of 1 (0 dB) then
converting to decibels
W_db = 20*log10(mag_W/max(mag_W));
% Frequency Vector
f = (0:1024-1)*(2/1024);
```

Furthermore, each window function is subjected to different window lengths to determine its effect on the width of the main lobe and the peak of the sidelobes.

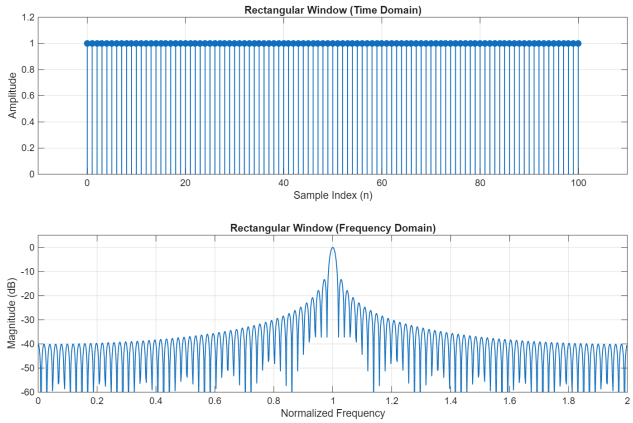

FIGURE 2.1. RECTANGULAR WINDOW TIME AND FREQUENCY DOMAIN FOR THE WINDOW LENGTH = 101 SAMPLES. THE FREQUENCY DOMAIN AMPLITUDE AXIS IS IN DECIBELS (DB) AND THE HORIZONTAL AXIS IS NORMALIZED.

TABLE 1. RECTANGULAR WINDOW.

L = length	W = main lobe width	K = (L)(W)	Peak Sidelobe (dB)
101	0.019536	1.973136	-13.3665
201	0.011719	2.355519	-13.401
301	0.007816	2.352616	-13.3267
401	0.007816	3.134216	-14.0313
501	0.003903	1.955403	-13.3206

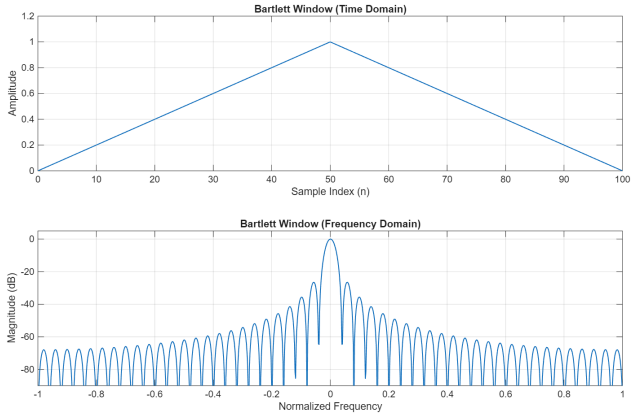

FIGURE 2.2 BARTLETT WINDOW TIME AND FREQUENCY DOMAIN FOR THE WINDOW LENGTH = 101 SAMPLES. THE FREQUENCY DOMAIN AMPLITUDE AXIS IS IN DECIBELS (DB) AND THE HORIZONTAL AXIS IS NORMALIZED.

TABLE 2. BARTLETT WINDOW.

L = length	W = main lobe width	K = (L)(W)	Peak Sidelobe (dB)
101	0.031245	3.155745	-26.5176
201	0.015622	3.140022	-26.6177
301	0.011719	3.527419	-26.6211
401	0.007816	3.134216	-26.8759
501	0.007816	3.915816	-26.6228

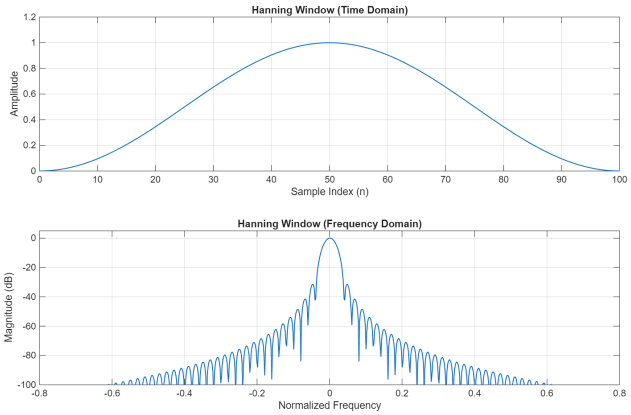

FIGURE 2.3 HANNING WINDOW TIME AND FREQUENCY DOMAIN FOR THE WINDOW LENGTH = 101 SAMPLES. THE FREQUENCY DOMAIN AMPLITUDE AXIS IS IN DECIBELS (DB) AND THE HORIZONTAL AXIS IS NORMALIZED.

TABLE 3. HANNING WINDOW.

L = length	W = main lobe width	K = (L)(W)	Peak Sidelobe (dB)
101	0.031245	3.155745	-31.4837
201	0.015622	3.140022	-31.4837
301	0.011719	3.527419	-31.4837
401	0.007816	3.134216	-31.4837
501	0.007816	3.915816	-31.7542

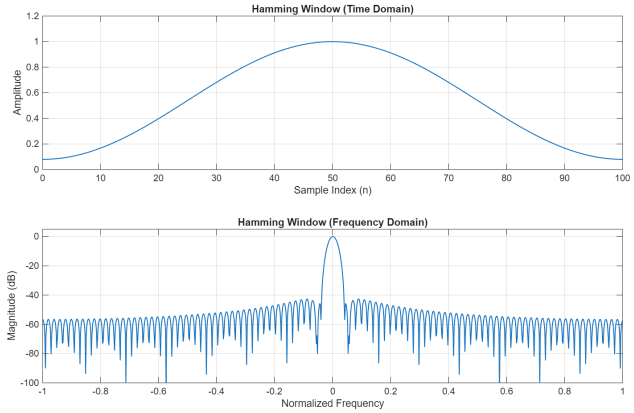

FIGURE 2.4 HAMMING WINDOW TIME AND FREQUENCY DOMAIN FOR THE WINDOW LENGTH = 101 SAMPLES. THE FREQUENCY DOMAIN AMPLITUDE AXIS IS IN DECIBELS (DB) AND THE HORIZONTAL AXIS IS NORMALIZED.

TABLE 4. HAMMING WINDOW.

L = length	W = main lobe width	K = (L)(W)	Peak Sidelobe (dB)
101	0.031245	3.155745	-42.6536
201	0.015622	3.140022	-42.6819
301	0.011719	3.527419	-42.9615
401	0.007816	3.134216	-43.2913
501	0.007816	3.915816	-43.033

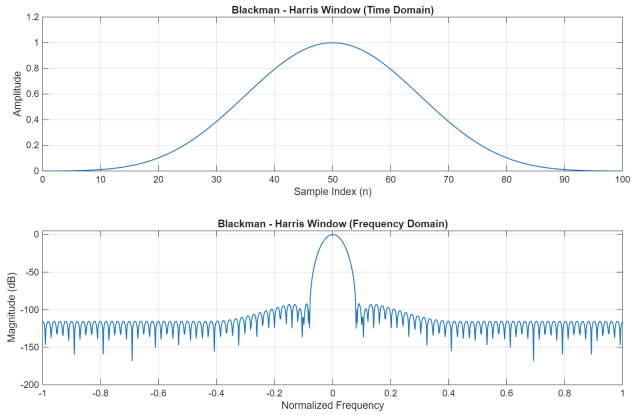

FIGURE 2.5 BLACKMAN - HARRIS WINDOW TIME AND FREQUENCY DOMAIN FOR THE WINDOW LENGTH = 101 SAMPLES. THE FREQUENCY DOMAIN AMPLITUDE AXIS IS IN DECIBELS (DB) AND THE HORIZONTAL AXIS IS NORMALIZED.

TABLE 5. BLACKMAN - HARRIS WINDOW.

L = length	W = main lobe width	K = (L)(W)	Peak Sidelobe (dB)
101	0.042964	4.339364	-92.075
201	0.019536	3.926736	-92.0639
301	0.015622	4.702222	-93.2311
401	0.011719	4.699319	-93.3211
501	0.007816	3.915816	-94.3841

II. Analysis and Discussion

Starting with the Rectangular window function, seen in figure 2.1. Its function has a sharp on and off transition. The effect of this abrupt transition can be seen in the frequency domain wherein there is a sharp main lobe meaning the frequency resolution is excellent but this comes at the price where the sidelobes are also very strong which can be seen in table 1 where the peak sidelobe value is at -13.3665 dB. Having strong sidelobes is not ideal for spectral analysis because it results in leakage. To reduce leakage, alternative window functions can be used such as the Bartlett window function or the Triangular window function as shown in figure 2.2. For this function, it has a smoother transition unlike the sharp transition the rectangular window function has. Having that smoother transition reduced the peak sidelobe to -26.5176 dB as seen in table 2. Because of the triangular shape of the window, the frequency resolution is worse than the frequency resolution of the rectangular window. If a signal is passed through the Bartlett window in the time domain, since the signal takes the shape of the window. This means, the amplitude of the samples at the edges will be reduced which then lowers the total energy of the signal when analyzed. This is the crucial trade-off the Bartlett window function offers. This is why a lot of window functions have been developed throughout the years to improve the trade-off the process of windowing has to offer.

Like the Hanning window in figure 2.3, wherein the main lobe is wider compared to the rectangular window but unlike the rectangular window, and its sidelobe is much lower as seen in table 3, which is equivalent to -31.4837 dB. Another window closely similar to the Hanning window is the Hamming window shown in figure 2.4, where it can be seen that the Hamming window has a thinner main lobe compared to the Hanning window but its sidelobes are much better which is equivalent to -42.6536 dB as shown in table 4. Lastly, the Blackman - Harris window function as seen in figure 2.5. It has the widest main lobe compared to the other four

windows mentioned above, and it has excellent sidelobe suppression as well wherein the peak sidelobe is measured at -92.075 dB in table 5.

The effect of the window length in relation to the performance of each window function as seen in tables 1 to 5. Increasing the window length has little to no difference when it comes to the peak sidelobes of each window function. But, it was observed that increasing the length of the window also increases the frequency oscillations thus making the main lobe width thinner.

III. Conclusion

Having different unique window functions is incredibly useful in spectral analysis. Each one has its own uses despite the trade-off that comes with it. For instance, the rectangular window is the simplest one and provides the narrowest main lobe which is important for the frequency resolution. Its drawback is that it also forms the worst sidelobes compared to other window functions. It is very important to consider the sidelobes as it can cause leakage and distortions on the analyzed signal. Considering this factor when choosing the window type, there are other alternative window functions that can be used to reduce the sidelobes that the rectangular window cannot provide, but just like the rectangular window every window function has its own drawback. Like the Bartlett window for example, it may be an improvement when it comes to reduced sidelobes but it comes at the cost of a wider main lobe. Now we know that by changing the window shape, a certain balance can be achieved depending on the situation. For instance, the Hanning and Hamming window can provide a thinner main lobe compared to the Blackman - Harris window but the Blackman - Harris is certainly better when it comes to reduced sidelobes. This unique balance in property is different for every window function but each one has its own uses when it comes to spectral analysis. It is important to know how each one performs to determine the right window to choose from to get an accurate frequency domain analysis.

FILTERING CAPABILITIES OF A FIRST-ORDER DT SYSTEM

Given the equation $y(n) = ay(n-1) + bx(n)$It represents a simple passive filter with one single real pole and a single zero at the origin of the unit circle. For this implementation, it will be designed as a passive filter that has a maximum nominal value of 1 or 0dB. A unity gain ensures that the system will be able to alter the frequency components without amplifying the input signal.

I. Implementation

A. Deriving for the Transfer Function

Given the equation below, where b and a are real numbers.

$$y(n) = ay(n-1) + bx(n)$$

The transfer function of the equation can be obtained by performing Z-Transform.

$$Y(z)) = az^{-1}Y(z) + bX(z)$$

Separate the Y(z) and X(z) variables and simplify the equation.

$$Y(z) - az^{-1}Y(z) = bX(z)$$

$$Y(z)\,(1 - az^{-1}) = bX(z)$$

$$\frac{Y(z)\,(1-az^{-1})}{(1-az^{-1})} = \frac{bX(z)}{(1-az^{-1})}$$

$$Y(z) = \frac{bX(z)}{(1-az^{-1})}$$

By isolating Y(z) and X(z) the transfer function H(z) can be obtained.

$$\frac{Y(z)}{X(z)} = \frac{bX(z)}{(1-az^{-1})(X(z))}$$

Hence the derived transfer function equation is:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{b}{1-az^{-1}}$$

B. Obtaining the DTFT $H(\omega)$ in terms of a and b.

To obtain the DTFT $H(\omega)$, we can substitute $z = r\,e^{jw}$. But since for this purpose, we only need to determine the filters' characteristic, r = 1 will be used for a constant magnitude.

$$H(z) = \frac{b}{(1-az^{-1})}$$

Substituting z,

$$H(\omega) = \frac{b}{1-ae^{-jw}}$$

C. Magnitude Response $|H(\omega)|$ in terms of a and b

$$H(\omega) = \frac{b}{(1-ae^{-jw})}$$

Given Euler's identity,

$$e^{jw} = cos(w) - jsin(w)$$

$$H(\omega) = \frac{b}{(1-a(cos(w)-jsin(w))}$$

$$|H(\omega)| = |\frac{b}{1-a(cos(\omega)-jsin(w))}|$$

$$|H(\omega)| = |\frac{b}{1-acos(w)-jasin(w)}|$$

Since the magnitude of a complex number is:

$$\sqrt{Real^2 + Imaginary^2}$$

$$|H(\omega)| = |\frac{1}{\sqrt{(1-acos(\omega)^2)+(asin(w))^2}}|$$

$$|H(\omega)| = |\frac{1}{\sqrt{(1-2acos(\omega)+a^2cos^2(\omega)+(a^2sin^2(w))}}|$$

$$|H(\omega)| = |\frac{1}{\sqrt{1-2acos(\omega)+a^2(cos^2(\omega)+sin^2(\omega))}}|$$

Given the trigonometric identity,

$$cos^2(\omega) + sin^2(\omega) = 1$$

Therefore,

$$|H(\omega)| = \frac{b}{\sqrt{1+a^2-2acos(\omega)}}$$

D. Condition for the maximum magnitude response

Based on the derived equation, the numerator b is directly proportional to the magnitude response $H(\omega)$ which acts as a magnitude scaling factor. For cos(ω), it is the one that dictates the maximum magnitude response. When cos(ω) = 1, a higher magnitude will be reached. Meanwhile, as variable a approaches 1, it decreases the value of the denominator which increases the overall magnitude. Overall, the maximum magnitude mostly depends on the value of the denominator.

E. Correct expression for $|H(\omega)|_{max}$, value of ω and type of filter

The previously derived expression was the correct expression for $|H(\omega)|_{max}$ since the max value of the happens when the value of the denominator is minimum.

Assuming a > 0:

a= 1

$$|H(\omega)|_{max} = \frac{b}{\sqrt{1+a^2-2acos(1)}}$$

$$|H(\omega)|_{max} = \frac{b}{\sqrt{1+a^2-2a(1)}}$$

$$|H(\omega)|_{max} = \frac{b}{\sqrt{(1-a)^2}}$$

$$|H(\omega)|_{max} = \frac{b}{1-a}$$

When $cos(\omega)$ is equal to 1, a higher magnitude is reached, which proves that the maximum magnitude mostly depends on the value of the denominator. The system yields a maximum magnitude when $cos(\omega)$ equals 1, which happens at ω= 0. This indicates that it is a Low-Pass Filter.

F. Determining the relationship between a and b

Since the focus of the problem is to design a passive filter, the overall transfer function must be equal to one. In order to attain the passive filter goal, there are two possible relationships between a and b. The first possible relationship is when the system is designed as a Low Pass Filter which peaks at the lowest frequency ($\omega = 0$) where the denominator is simplified into $(1 - a)$. The goal is to make $|H(\omega)|_{max}$ =1.

Therefore, the relationship is:

$$b = 1 - a.$$

On the other hand, when the system is designed as a High Pass Filter, the response is peak at the highest frequency ($\omega = \pi$) wherein the $cos(\omega)$ value becomes -1 which makes the denominator simplify to just $(1 + a)$.

Therefore, the relationship is:

$$b = 1 + a$$

Ultimately, in order to attain $|H(\omega)|_{\max}$ =1, the value of the numerator needs to be similar with the denominator which will lead to a balanced ratio.

G. Design Equation

In order to design this Low Pass Filter, the cutoff frequency (ωc) of a filter must be at the frequency at which the magnitude response is $1/\sqrt{2}$ or 70.71% of its maximum value.

$$|H(\omega c)| = |H(\omega)|_{max}/\sqrt{2}$$

By using the passive filter relationships in the magnitude equation and performing simplifications, a quadratic equation is formed which will lead to two possible equations for coefficient a.

Substituting the Low Pass Filter b and a relationship:

$$\frac{(1-a)^2}{1+a^2-2a\cos(\omega c)} = \frac{1}{2}$$

By cross-multiplying,

$$2(1-a)^2 = 1(1+a^2-2a\cos(\omega c))$$

$$2(1-2a+a^2) = 1+a^2-2a\cos(\omega c)$$

$$2-4a+2a^2 = 1+a^2-2a\cos(\omega c)$$

Rearranging,

$$2a^2-a^2-4a+2a\cos(\omega c)+2-1=0$$

$$a^2-2a(2-\cos(\omega c))+1=0$$

Using the quadratic formula,

$$a = \frac{-B\pm\sqrt{B^2-4AC}}{2A}$$

$$a = \frac{-[-2(2-\cos(\omega c)]]\pm\sqrt{[-2(2-\cos(\omega c)]]^2-4(1)(1)}}{2(1)}$$

$$a = \frac{2(2-\cos(\omega c)]]\pm\sqrt{4(2-\cos(\omega c)^2-4}}{2}$$

$$a = \frac{2(2-\cos(\omega c)]]\sqrt{4}\pm\sqrt{(2-\cos(\omega c)^2-1}}{2}$$

$$a = \frac{2(2-\cos(\omega c)]]\,2\pm\sqrt{(2-\cos(\omega c)^2-1}}{2}$$

$$a = (2-\cos(\omega c))\pm\sqrt{(2-\cos(\omega c))^2-1}$$

To ensure that the system is stable, the magnitude of the feedback coefficient a must be less than 1 ($|a|<1$). Using the plus sign results in a value greater than 1. Therefore, we must choose the minus sign in the 2 equations.

$$a = (2-\cos(\omega c))-\sqrt{(2-\cos(\omega c))^2-1}$$

H. MATLAB Implementation of Derived Equations

To implement the derived equations in MATLAB, a function named 'FirstOrdPass' was created. This function is able to take an angular cutoff frequency (wc) as its input and the pole and zero coefficients (b & a) as its output. Based on the previous equations a relationship of $b = 1 - a$ was used as it will provide a unity gain for the filter.

MATLAB CODE:

```
function [b, a] = FirstOrdPass(wc)

% from the derived equation
a = (2-cos(wc))-sqrt((2-cos(wc)).^2-1);

% relationship of b & a
b = 1 - a;

% rearrange a to its first order format
a = [1 -a];

end
```

I. Determining Filter Characteristics

To determine the filter characteristics of the first-order filter, the following angular cutoff frequencies were used to the determine the magnitude response:

$$\omega_{c1} = 0.1\pi$$
$$\omega_{c2} = 0.2\pi$$
$$\omega_{c3} = 0.3\pi$$

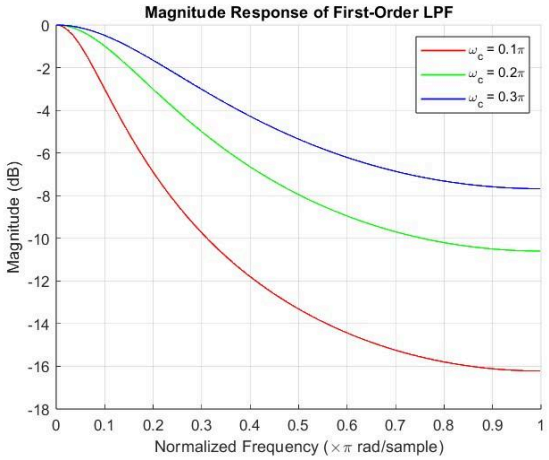

FIGURE 3.1 FILTER RESPONSE USING DIFFERENT ANGULAR FREQUENCIES

J. Filter Testing

To test the effectiveness of the created filter, the filter was applied to a given noise signal 'NoisyTones.wav'. Given the first order equation , the group was able to derive the filter transfer function using Z-transform. In the Z-domain, the frequencies are determined on the unit circle as $z = re^{jw}$, where the angle represents the normalized frequency. By using the 'FirstOrdPass' function, the pole and zero (b and a) locations can be determined. The designed filter was then applied using MATLAB and it's built in 'filter command.

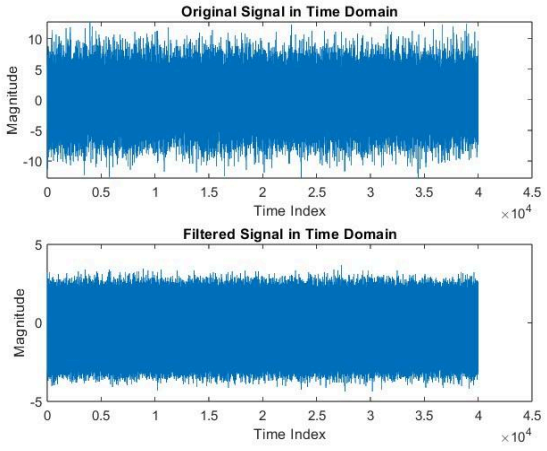

FIGURE 3.2 COMPARISON OF ORIGINAL AND FILTERED SIGNAL IN TIME DOMAIN

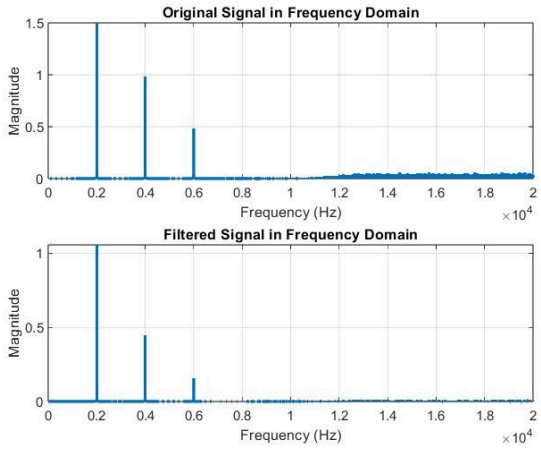

FIGURE 3.3 COMPARISON OF ORIGINAL AND FILTERED SIGNAL IN FREQUENCY DOMAIN

In order to audibly hear the effect of the filter, the filtered signal was saved as a wav file using the following MATLAB command:

```
audiowrite('CleanTones.wav', y, fs);
```

II. Analysis and Discussion

As observed in Figure 3.1, its resulting magnitude responses showed the characteristic of a low pass filter. The passband also proves this further, by having a higher magnitude at lower frequencies then drops lower as frequencies increase. More importantly, it satisfied the condition of a maximum nominal value of 1 or a decibel value of 0 dB for all tested values of angular frequency.

.

As the angular frequency increases 0.1π to 0.3π, the roll-off becomes more shallow. Furthermore, as the angular frequency increases, its attenuation also decreases significantly. At 0.1π, the highest attenuation reached around -16.2dB whereas the 0.2π and 0.3π only reached around -10.6dB and -7.67db of attenuation.

As shown in Figure 3.2, after applying the filter to the input signal, the highest magnitude of the signal in the time domain was significantly decreased. Furthermore, the peaks from the magnitude which was scattered at the original signal, was now much more even in the filtered signal. On the other hand, when plotting the frequency domain, Figure 3.3 shows that the noise in higher frequencies was attenuated. Overall, it can be said that the first order system was able to successfully filter the higher frequency signals.

III. Conclusion

In conclusion, the designed filter was successfully transformed into a functional passive low-pass filter. By deriving the transfer function and, the design ensured unity gain, allowing the system to process signals without introducing instability or unwanted amplification. The formulation of the design equation further allowed for the calculation of coefficient a, based on the desired angular cutoff frequency.

The testing phase exhibited the filter's performance, where the direct correlation between the cutoff frequency and the filter's attenuation capabilities was successfully shown. As observed in the frequency domain analysis, increasing the angular cutoff frequency resulted in a shallower roll-off and decreased attenuation of high-frequency components. This behavior was confirmed during the processing of the "NoisyTones.wav" signal in which the filter successfully suppressed high-frequency noise and reduced the signal amplitude in the time domain. Although the audible changes were subtle, the spectral data confirmed that the system functioned correctly as a stable, passive low-pass filter suitable for basic noise reduction tasks.

References

[1] John G. Proakis and Dimitris G. Manolakis. *Digital Signal Processing: Principles, Algorithms, and Application*. Prentice-Hall International, 2016

[2] Vinay K. Ingle and John G. Proakis. *Digital Signal Processing using MATLAB, 3rd Edition*. Cengage Learning, 2012.

[3] Steven Smith. *The Scientist and Engineer's Guide to Digital Signal Processing, 2nd Edition*. California Technical Publishing, 1999.

Distribution of Tasks

Members	Tasks
MABALLO, Ralph Vincent E.	1. Introduction 2. Digital Oscillator Design 3. Implementation 4. Discussion and Analysis
DELA CHICA, Kyle EJ C.	1. Window Function Evaluation 2. Introduction 3. Abstract
MANANGUIT, Jared Asher N.	1. Filtering Capabilities of a First-Order DT System 2. Implementation (MATLAB Coding) 3. Analysis and Conclusion
PANGAN, Raven G.	1. Filtering Capabilities of a First-Order DT System 2. Implementation (Derivations)

Appendix A

```
% Number of samples
N = 50;
n = 0:N-1;
% Impulse input, x(n)=delta(n)
x = [1 zeros(1,N-1)];
% Frequencies of each subsystem
w1 = 3*pi/10;
w2 = pi/2;
w3 = 3*pi/4;
% Coefficients of subsystem 1
b1 = [1 -cos(w1)];
a1 = [1 -2*cos(w1) 1];
% Coefficients of subsystem 2
b2 = [1 -cos(w2)];
a2 = [1 -2*cos(w2) 1];
% Coefficients of subsystem 3
b3 = [1 -cos(w3)];
a3 = [1 -2*cos(w3) 1];
% %Calculate Total Denominator (Convolution of
all denominators)
% A_total = a1 * a2 * a3
a12 = conv(a1, a2);
a_total = conv(a12, a3);
% B_total = (b1*a2*a3) + (b2*a1*a3) +
(b3*a1*a2);
% Calculate the Total Numerator
(Cross-multiplication sum)
term1 = conv(b1, conv(a2, a3));
term2 = conv(b2, conv(a1, a3));
term3 = conv(b3, conv(a1, a2));
b_total = term1 + term2 + term3;
%Simulate Parallel Branches
y_p1 = filter(b1,a1,x);
y_p2 = filter(b2,a2,x);
y_p3 = filter(b3,a3,x);
%Sum for Final Parallel Output
y_parallel = y_p1 + y_p2 + y_p3;
% Factoring the consolidated 6th-order
% transfer function
```

```
% into Second-Order Sections(SOS).
[sos_matrix, gain] = tf2sos(b_total, a_total);
y_cascade = x * gain; % Applying the overall
gain
% Loop through each 2nd-order section
for k = 1:size(sos_matrix, 1)
b_k = sos_matrix(k, 1:3); % Extract numerators
a_k = sos_matrix(k, 4:6); % Extract
denominators
% The input to this stage is the output of the
previous stage
y_cascade = filter(b_k, a_k, y_cascade);
end
y_stf = filter(b_total, a_total, x);
% Time Domain Plots
% Parallel Second-Order Subsystem
figure;
stem(n,y_parallel,'filled');
xlabel('n');ylabel('y(n)');
title('Parallel Realization Output (Time
Domain)');
grid on;
% Cascaded Subsystems (The sections from
Second-Order Subsystem)
figure;
stem(n, y_cascade, 'filled');
xlabel('n');ylabel('y(n)');
title('Cascaded Realization Output (Time
Domain)');
grid on;
% Single Transfer Function (The 6th Order
System)
figure;
stem(n, y_stf, 'filled');
xlabel('n');ylabel('y(n)');
title("Single Transfer Function Output(Time
Domain)");
grid on;
% Frequency Domain Plots
N = 1024;
Fs = 2; % Set Nyquist (Fs/2)=1 to represent pi
rad/sample;
% Frequency Vector
N_vector = (0:N/2-1)/(N/2-1)*Fs/2;
% Compute FFT (keep only the first half)
Y_n = fft(y_stf, N);
Y_n = Y_n(1:N/2);
figure;
plot(N_vector, abs(Y_n));
title('Digital Oscillator Frequency Response');
xlabel('Normalized Frequency (\pi
rad/sample)');
ylabel('|Y(\omega)|');
grid on;
%Pole-Zero Plots
% Parallel Second-Order Subsystem
figure;
subplot(1,3,1); zplane(b1, a1);
title('Subsystem1');
subplot(1,3,2); zplane(b2, a2);
title('Subsystem2');
subplot(1,3,3); zplane(b3, a3);
title('Subsystem3');
sgtitle('Pole-Zero Plots: Parallel Second-Order
Subsystem');
% Cascaded Subsystems (The sections from
Second-Order Subsystem)
figure;
subplot(1,3,1); zplane(sos_matrix(1,1:3),
sos_matrix(1,4:6)); title('Cascade Stage 1');
subplot(1,3,2); zplane(sos_matrix(2,1:3),
sos_matrix(2,4:6)); title('Cascade Stage 2');
subplot(1,3,3); zplane(sos_matrix(3,1:3),
sos_matrix(3,4:6)); title('Cascade Stage 3');
sgtitle('Pole-Zero Plot: Cascaded Second-Order
Subsystem');
% Single Transfer Function (The 6th Order
System)
figure;
zplane(b_total, a_total);
title('Pole-Zero Plot: Single Transfer Function
(6th Order)');
```

Appendix B

```
  %% Rectangular Window Function
  % Parameters
  L = 101;  % Window Length
  n = 0 : L-1;  % Sample Index
  % Implementation of the Rectangular Window
Function
  w = ones(L,1);
  % Computation for the Frequency Response
  W = fft(w,1024); % 1024 - point FFT
  W_shifted = fftshift(W); % Shift zero
frequency to center
  % Normalize Magnitude
  mag_W = abs(W_shifted);
  % Normalizing to peak of 1 (0 dB) then
converting to decibels
  W_db = 20*log10(mag_W/max(mag_W));
  % Frequency Vector
  f = (0:1024-1)*(2/1024);
  % Create Figure
  figure('Name','Rectangular Window
Evaluation');
  % Plotting Time Domain
  subplot(2,1,1);
  stem(n,w,'filled',’LineWidth',1);  % Time
domain plot
  title('Rectangular Window (Time Domain)'); %
Title for time domain plot
  xlabel('Sample Index (n)'); % Label for the
x axis
  ylabel('Amplitude'); % Label for the y axis
  xlim([-10 110]); % Setting x - limit for
aesthetics and clarity
  ylim([0 1.2]); % Setting y - limit for
aesthetics and clarity
  grid on;
  % Plotting Frequency Domain
  subplot(2,1,2);
  plot(f, W_db,'LineWidth',1); % Frequency
domain plot
  title('Rectangular Window (Frequency
Domain)'); % Title for frequency domain
  xlabel('Normalized Frequency'); % Label for
the x axis
  ylabel('Magnitude (dB)'); % Label for the y
axis
  ylim([-60 5]); % Setting y - limit for
aesthetics and clarity
  grid on;

  %% Bartlett Window Function
  % Parameters
  L = 101; % Window Length
  n = 0 : L-1; % Sample Index
```

```
% Implementation of the Bartlett Window Function
N = 2 * abs(n - (L-1)/2);
w = 1 - N / (L - 1);
% Computation for the Frequency Response
W = fft(w,1024); % 1024 - point FFT
W_shifted = fftshift(W); % Shift zero frequency to center
% Normalize Magnitude
mag_W = abs(W_shifted);
% Normalizing to peak of 1 (0 dB) then converting to decibels
W_db = 20*log10(mag_W/max(mag_W));
% Frequency Vector
f = (0:1024-1)*(2/1024);
% Create Figure
figure('Name','Bartlett (Triangular) Window Evaluation');
% Plotting Time Domain
subplot(2,1,1);
plot(n,w,'LineWidth',1); % Time domain plot
title('Bartlett Window (Time Domain)'); % Title for time domain plot
xlabel('Sample Index (n)'); % Label for the x axis
ylabel('Amplitude'); % Label for the y axis
ylim([0 1.2]); % Setting y - limit for aesthetics and clarity
grid on;
% Plotting Frequency Domain
subplot(2,1,2);
plot(f,W_db,'LineWidth',1); % Frequency domain plot
title('Bartlett Window (Frequency Domain)'); % Title for frequency domain
xlabel('Normalized Frequency'); % Label for the x axis
ylabel('Magnitude (dB)'); % Label for the y axis
ylim([-90 5]); % Setting y - limit for aesthetics and clarity
grid on;

%% Hanning Window Function
% Parameters
L = 101; % Window Length
n = 0 : L-1; % Sample Index
% Implementation of the Hanning Window Function
w = 0.5 * (1 - cos(2 * pi * n / (L-1)));
% Computation for the Frequency Response
W = fft(w,1024); % 1024 - point FFT
W_shifted = fftshift(W); % Shift zero frequency to center
% Normalize Magnitude
mag_W = abs(W_shifted);
% Normalizing to peak of 1 (0 dB) then converting to decibels
W_db = 20 * log10(mag_W/max(mag_W));
% Frequency Vector
f = (0:1024-1)*(2/1024);
% Create Figure
figure('Name','Hanning Window Evaluation');
% Plotting Time Domain
subplot(2,1,1);
plot(n,w,'LineWidth',1); % Time domain plot
title('Hanning Window (Time Domain)'); % Title for time domain plot
xlabel('Sample Index (n)'); % Label for the x axis
ylabel('Amplitude'); % Label for the y axis
ylim([0 1.2]); % Setting y - limit for aesthetics and clarity
grid on;
% Plotting Frequency Domain
subplot(2,1,2);
plot(f,W_db,'LineWidth',1); % Frequency domain plot
title('Hanning Window (Frequency Domain)'); % Title for frequency domain
xlabel('Normalized Frequency'); % Label for the x axis
ylabel('Magnitude (dB)'); % Label for the y axis
ylim([-100 5]); % Setting y - limit for aesthetics and clarity
grid on;

%% Hamming Window Function
% Parameters
L = 101; % Window Length
n = 0 : L-1; % Sample Index
% Implementation of the Hamming Window Function
w = 0.54 - 0.46 * cos(2 * pi * n / (L-1));
% Computation for the Frequency Response
W = fft(w,1024); % 1024 - point FFT
W_shifted = fftshift(W); % Shift zero frequency to center
% Normalize Magnitude
mag_W = abs(W_shifted);
% Normalizing to peak of 1 (0 dB) then converting to decibels
W_db = 20 * log10(mag_W/max(mag_W));
% Frequency Vector
f = (0:1024-1)*(2/1024);
% Create Figure
figure('Name','Hamming Window Evaluation');
% Plotting Time Domain
subplot(2,1,1);
plot(n,w,'LineWidth',1); % Time domain plot
title('Hamming Window (Time Domain)'); % Title for time domain plot
xlabel('Sample Index (n)'); % Label for the x axis
ylabel('Amplitude'); % Label for the y axis
ylim([0 1.2]); % Setting y - limit for aesthetics and clarity
grid on;
% Plotting Frequency Domain
subplot(2,1,2);
plot(f,W_db,'LineWidth',1); % Frequency domain plot
title('Hamming Window (Frequency Domain)'); % Title for frequency domain
xlabel('Normalized Frequency'); % Label for the x axis
ylabel('Magnitude (dB)'); % Label for the y axis
ylim([-100 5]); % Setting y - limit for aesthetics and clarity
grid on;

%% Blackman - Harris Window Function
% Parameters
L = 101; % Window Length
n = 0 : L-1; % Sample Index
% Implementation of the Blackman - Harris Window Function
a0 = 0.35875;
a1 = 0.48829 ; % Coefficient a1
a2 = 0.14128 ; % Coefficient a2
```

```
   a3 = 0.01168 ; % Coefficient a3
   % General Equation for the Blackman - Harris
Window
   w = a0 - ...
      a1 * cos(2 * pi * n/ (L-1)) + ...
      a2 * cos(4 * pi * n/ (L-1)) - ...
      a3 * cos(6 * pi * n/ (L-1));
   % Computation for the Frequency Response
   W = fft(w,1024); % 1024 - point FFT
   W_shifted = fftshift(W); % Shift zero
frequency to center
   % Normalize Magnitude
   mag_W = abs(W_shifted);
   % Normalizing to peak of 1 (0 dB) then
converting to decibels
   W_db = 20 * log10(mag_W/max(mag_W));
   % Frequency Vector
   f = (0:1024-1)*(2/1024);
   % Create Figure
   figure('Name','Blackman - Harris Window
Evaluation');
   % Plotting Time Domain
   subplot(2,1,1);
   plot(n,w,'LineWidth',1); % Time domain plot
   title('Blackman - Harris Window (Time
Domain)'); % Title for time domain plot
   xlabel('Sample Index (n)'); % Label for the
x axis
   ylabel('Amplitude'); % Label for the y axis
   ylim([0 1.2]); % Setting y - limit for
aesthetics and clarity
   grid on;
   % Plotting Frequency Domain
   subplot(2,1,2);
   plot(f,W_db,'LineWidth',1); % Frequency
domain plot
   title('Blackman - Harris Window (Frequency
Domain)'); % Title for frequency domain
   xlabel('Normalized Frequency'); % Label for
the x axis
   ylabel('Magnitude (dB)'); % Label for the y
axis
   ylim([-200 5]); % Setting y - limit for
aesthetics and clarity
   grid on;
```

Appendix C

Function for First Order Poles and Zeros

```
function [b, a] = FirstOrdPass(wc)

% from the derived equation
a = (2-cos(wc))-sqrt((2-cos(wc)).^2-1);
% relationship of b & a
b = 1 - a;
% rearrange a to its first order format
a = [1 -a];
end
```

Plotting the frequency domain of given cutoff frequencies

```
%Defining natural cutoff frequencies
wc1 = 0.1*pi;
wc2 = 0.2*pi;
wc3 = 0.3*pi;
% extra coefficients using created function
[b1, a1] = FirstOrdPass(wc1);
[b2, a2] = FirstOrdPass(wc2);
[b3, a3] = FirstOrdPass(wc3);
% determine the frequency response
[H1, w1] = freqz(b1, a1, 1024);
[H2, w2] = freqz(b2, a2, 1024);
[H3, w3] = freqz(b3, a3, 1024);
figure;
hold on;
grid on;
%plot in dB unit
plot(w1/pi, 20*log10(abs(H1)), 'r',
'LineWidth', 1);
plot(w2/pi, 20*log10(abs(H2)), 'g',
'LineWidth', 1);
plot(w3/pi, 20*log10(abs(H3)), 'b',
'LineWidth', 1);
title('Magnitude Response of First-Order LPF');
xlabel('Normalized Frequency (\times\pi
rad/sample)');
ylabel('Magnitude (dB)');
legend('\omega_c = 0.1\pi', '\omega_c =
0.2\pi', '\omega_c = 0.3\pi');
```

Plotting Time Domain and Frequency Domain Responses of Original and Filtered signal

```
%Loading input signal
[x, fs] = audioread('NoisyTones.wav');
% converting input signal to frequency domain
N = length(x);
xf = abs(fft(x))/N;
%define the cutoff frequency
wc = 0.1 * pi;
[b, a] = FirstOrdPass(wc);
%applying the filter on input signal
y = filter(b, a, x);
yf = abs(fft(y))/N;
figure;
%plotting original and filtered signals in time
domain
subplot(2,1,1);
plot(x);
title('Original Signal in Time Domain');
xlabel('Time Index');
ylabel('Magnitude');
subplot(2,1,2);
plot(y);
title('Filtered Signal in Time Domain');
xlabel('Time Index');
ylabel('Magnitude');
figure;
%plotting original and filtered signals in
frequency domain
subplot(2,1,1)
plot(xf(1:N/2), LineWidth=2);
title('Original Signal in Frequency Domain')
xlabel('Frequency (Hz)'); ylabel('Magnitude');
grid on;
subplot(2,1,2);
plot(yf(1:N/2), LineWidth=2);
title('Filtered Signal in Frequency Domain');
xlabel('Frequency (Hz)'); ylabel('Magnitude');
grid on;
```

Saving the filtered signal to .wav file

```
% 1. Read the audio file
[x, fs] = audioread('NoisyTones.wav');
N = length(x);
% defining the cutoff frequency
```

```
wc = 0.1 * pi;
[b, a] = FirstOrdPass(wc);
%applying the filter function
y = filter(b, a, x);
y = y/max(abs(y));
%normalize filtered signal in frequency domain
yf = abs(fft(y))/N;
%saving the output using audiowrite function
audiowrite('CleanTones.wav', y, fs);
```

Appendix D

I acknowledge the use of Google Gemini, (https://gemini.google.com/app) to generate an outline on how to start my research and the general information I need for the research for it to be coherent and comprehensive. I entered the following prompt on December 11, 2025:

- I am tasked to research windowing techniques in DSP, specifically five of them (Rectangular, Barlett, Hanning, Hamming, and Blackman-Harris). Can you outline for me how I can go about my research. What do I need to look for to better understand these techniques and what I need to put on my research for it to be coherent and comprehensive.

The output from these prompts was used to gain insight on what information to look for and how to structure the paper.

Kyle EJ C. dela Chica
3ECE - B

I acknowledge the use of Google Gemini (https://gemini.google.com/app) to generate an outline on how to start my research and the general information I need for the research, for it to be coherent and comprehensive. I entered the following prompt on December 15, 2025:

- I am tasked to research and design a Digital Oscillator. I ask questions to Gemini:
  1. Can you outline for me how I can go about my research?
  2. What do I need to look for to better understand these techniques, and what do I need to put into my research for it to be coherent and comprehensive?
  3. How will I do the Cascaded system, where it functions to break the big system into smaller pairs and connect them in a series, so the signal goes through them one after another
     - Gemini suggested using a tool called "tf2sos."

The output from these prompts was used to gain insight into what information to look for and how to structure the paper

Ralph Vincent E.
Maballo
3ECE-B

I acknowledge the use of Google Gemini (https://gemini.google.com/app) to generate the correct and appropriate grammar needed, for formality and smooth flow of the readings. I entered the following prompt on December 17, 2025:

- "When cos() is equals to 1, higher magnitude will be reached which proves that the maximum magnitude mostly depends on the value of the denominator. The system yields a maximum magnitude when the cos(w) is equals to one which happens at w = 0. This depicts as a Low-Pass Filter. Check grammar"
- "Ultimately, in order to attain |H()|max =1, the value of the numerator needs to be similar with the denominator which will lead to a balanced ratio. Check grammr"
- "To ensure that the system is stable, the magnitude of the feedback coefficient a must be less than 1 (|a|<1).Using the plus sign will make the value greater than 1, therefore, we must choose the minus sign in the 2 equations. Check grammar"

Aside from the approval of groupmates for the derived equations, I also entered the following prompt for assurance of correct simplification.

- "Is my simplification correct"

The output from these prompts was used to make sure, improve and structure the equations and paragraphs in particular sections of this paper.

Raven G. Pangan
3ECE-B

Jared Asher N. Mananguit
3-ECEB

Gemini DSP Window...

I am tasked to research windowing techniques in DSP specifically five of them (Rectangular, Barlett, Hanning, Hamming, and Blackman-Harris). Can you outline for me how I can go about my research. What do I need to look for to better understand these techniques and what I need to put on my research for it to be coherent and comprehensive.

Show thinking

Researching windowing techniques is a fundamental rite of passage in Digital Signal Processing (DSP). To understand these specific five windows (Rectangular, Bartlett, Hanning, Hamming, and Blackman-Harris), you need to move beyond just their mathematical formulas and understand the **trade-offs** they represent.

Here is a comprehensive roadmap to guide your research, structured to help you build a coherent and high-quality report.

Phase 1: Understand the "Why" (The Problem)

Before diving into the specific windows, you must clearly define the problem they solve. Your research should start by defining **Spectral Leakage**.

- **The Concept:** When we take a Finite Fourier Transform (FFT) of a signal, we are taking a finite "chunk" of data. The FFT assumes this chunk repeats infinitely. If the signal doesn't perfectly fit in that window (i.e., end points don't match), it creates sharp discontinuities at the edges.

Enter a prompt for Gemini

Thinking

Your University of Santo Tomas chats aren't used to improve our models. Gemini can make mistakes, so double-check it. Your privacy & Gemini

$$a = \frac{2(2-\cos(\omega c))] \cdot \frac{4 \pm \sqrt{(2-\cos(\omega c)^2 - 1}}{2}}{}$$

$$a = \frac{2(2-\cos(\omega c))] \, 2 \pm \sqrt{(2-\cos(\omega c)^2 - 1}}{2}$$

$$a = (2 - \cos(\omega c)) \pm \sqrt{(2 - \cos(\omega c))^2 - 1}$$

is my simplification correct

Show thinking

Yes, your simplification is **correct**. Every step follows the algebraic rules of the quadratic formula and radical simplification.

Here is a breakdown of your steps to confirm:

When cos() is equals to 1, higher magnitude will be reached which proves that the maximum magnitude mostly depends on the value of the denominator. The system yields a maximum magnitude when the cos(w) is equals to one which happens at w = 0. This depicts as a Low-Pass Filter.

correct my grammar

Show thinking

Here are a few ways to correct and improve your paragraph, depending on how formal you want to be:

Option 1: Minimal Changes (Keeps your original structure)

"When $\cos(\omega)$ **is equal** to 1, a higher magnitude **is** reached, which proves that the maximum magnitude mostly depends on the value of the denominator. The system yields a maximum magnitude when $\cos(\omega)$ **equals** 1, which happens at $\omega = 0$. This **indicates that it is** a Low-Pass Filter."

Ultimately, in order to attain |H()|max =1, the value of the numerator needs to be similar with the denominator which will lead to a balanced ratio.

check grammar

Examining Sentence Structure

Here are a few options to improve the grammar and precision of your sentence:

Option 1: Most Accurate (Recommended for Engineering)

"Ultimately, in order to attain $|H(\omega)|_{\max} = 1$, the value of the numerator needs to be **equal to** the denominator, which will lead to a **ratio of one**."

To ensure that the system is stable, the magnitude of the feedback coefficient a must be less than 1 (|a| <1).Using the plus sign will make the value greater than 1, therefore, we must choose the minus sign in the 2 equations....

Show thinking

Here are a few ways to refine the grammar and flow of your sentence:

Option 1: Clear and Direct (Best for technical reports)

"To ensure system stability, the magnitude of the feedback coefficient a must be less than 1 ($|a| < 1$). Using the plus sign results in a value greater than 1; therefore, we must choose the minus sign from the two possible equations."

- **The Solution:** Windowing functions smooth out these edges (tapering the signal to zero at the boundaries) to reduce this leakage.

Phase 2: Key Metrics for Comparison

To make your research coherent, you cannot just describe the windows individually; you must **compare** them using a standard set of metrics. Look for these three specific characteristics for each window:

1. **Main Lobe Width:**
   - How "fat" is the central peak in the frequency domain?
   - *Significance:* A wider main lobe reduces **Frequency Resolution**. It makes it harder to distinguish two signals that are close together in frequency.
2. **Peak Side Lobe Level:**
   - How high is the first "ripple" next to the main peak? (Usually measured in decibels, dB).
   - *Significance:* High side lobes can hide low-amplitude signals present at other frequencies. You generally want this to be as low (negative dB) as possible.
3. **Side Lobe Roll-off Rate:**
   - How quickly do the ripples die down as you move away from the main frequency? (Measured in dB/octave or dB/decade).

The Golden Rule of Windowing: There is always a trade-off. You generally cannot have both a very narrow main lobe (high resolution) and very low side lobes (high dynamic range) simultaneously.

Phase 3: The Five Windows (What to look for)

Phase 3: The Five Windows (What to look for)

Here is a primer on what to look for regarding the specific five windows you are researching.

1. Rectangular Window

- **What it is:** This is effectively "no windowing." You just take the chunk of data as is.
- **The Math:** $w(n) = 1$
- **Characteristics:** It has the **narrowest main lobe** (best resolution) but the **worst spectral leakage** (highest side lobes, around -13 dB).
- **Best Use Case:** When you are analyzing transients or signals shorter than the window length, or when frequency resolution is paramount and noise is low.

2. Bartlett Window (Triangle)

- **What it is:** A simple triangular taper. It rises linearly from zero to one and falls back to zero.
- **The Math:** $w(n) = 1 - \left|\frac{n - \frac{N-1}{2}}{\frac{N-1}{2}}\right|$
- **Characteristics:** It reduces side lobes better than Rectangular (approx -26 dB) but widens the main lobe significantly. It is rarely used in modern applications compared to the ones below, but it is historically important.

3. Hanning (Hann) Window

- **What it is:** A "raised cosine" window. It is very smooth and touches zero at both ends.
- **The Math:** $w(n) = 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right)$
- **Characteristics:** A great general-purpose window. It has a good rollout rate and significantly lower side lobes (-31 dB) than Rectangular. It is arguably the most common window for random noise measurements.

4. Hamming Window

- **What it is:** A variation of the Hanning. It does *not* taper all the way to zero at the edges (it sits on a small pedestal).
- **The Math:** $w(n) = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right)$
- **Characteristics:** It is optimized to cancel out the first side lobe specifically. This gives it a lower peak side lobe (-43 dB) than Hanning, but the side lobes further out don't decay as fast (slower roll-off).

5. Blackman-Harris Window

- **What it is:** A more complex "sum of cosines" window.
- **Characteristics:** This is the heavy hitter for **Dynamic Range**. It has an extremely wide main lobe (bad resolution) but incredibly low side lobes (often below -92 dB).
- **Best Use Case:** When you need to find a tiny signal hiding next to a massive signal.

Phase 4: Structuring Your Research Paper

To make your report comprehensive, I suggest the following outline:

I. Introduction

- Definition of the DSP context (FFT analysis).
- Explanation of the problem: Spectral Leakage.

II. Mathematical Definitions

- Present the equations for all five windows clearly (use the LaTeX provided above as a reference).
- Show the Time-Domain plots (what the windows look like).

- Explanation of the problem: Spectral Leakage.

II. Mathematical Definitions

- Present the equations for all five windows clearly (use the LaTeX provided above as a reference).
- Show the Time-Domain plots (what the windows look like).

III. Frequency Domain Analysis (The Core)

- Compare the Frequency Response of each.
- **Create a Comparison Table:** This is crucial for a "coherent" report.
  - Columns: Window Name | Main Lobe Width (-3dB) | Peak Side Lobe (dB) | Roll-off Rate.

IV. Performance Trade-offs

- Discuss Resolution vs. Dynamic Range.
- Explain why one might choose Hamming over Blackman-Harris (or vice versa).

V. Application Scenarios

- Give examples (e.g., "Use Rectangular for separating two close frequencies of similar strength," "Use Blackman-Harris for detecting a weak signal near a strong interferer").

A Next Step for You

To truly understand these concepts, you need to see them visually. If you have access to Python or MATLAB, **would you like me to write a short script that generates the Time Domain and Frequency Response plots for these five specific windows so you can include them in your research?**